

code

March 27, 2026

0.1 IMPORTS

```
[90]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sentencepiece as spm

import os
import math
import time
import random
from pathlib import Path
from dataclasses import dataclass
from typing import List, Tuple, Dict

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, \
    confusion_matrix

from tqdm.auto import tqdm as tqdm_auto
from tqdm import tqdm as tqdm

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence

from torchinfo import summary
from torchview import draw_graph
import graphviz as gv
```

0.2 SEED

```
[91]: SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
```

```

if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

```

0.3 DEVICE SETUP

```

[92]: USE_CUDA = torch.cuda.is_available()
DEVICE = torch.device("cuda" if USE_CUDA else "cpu")
USE_AMP = USE_CUDA

if USE_CUDA:
    torch.backends.cudnn.benchmark = True
    torch.backends.cuda.matmul.allow_tf32 = True
    torch.backends.cudnn.allow_tf32 = True
    torch.set_float32_matmul_precision("high")

print(f"Device: {DEVICE}")
print(f"AMP enabled: {USE_AMP}")

if USE_CUDA:
    print(f"CUDA is available. Training will run on GPU.")
else:
    print("CUDA is not available in this kernel. Training will run on CPU and
    ↳be very slow.")

```

```

Device: cuda
AMP enabled: True
CUDA is available. Training will run on GPU.

```

0.4 PATHS

```

[93]: root = Path.cwd()
data_dir = root / 'Data'
corpus_dir = data_dir / 'hindi_corpus' / 'train'
cls_csv = data_dir / 'text_classification_dataset' / 'train.csv'

out_dir = root / 'outputs'
(out_dir / "checkpoints").mkdir(parents=True, exist_ok=True)
(out_dir / "generations").mkdir(parents=True, exist_ok=True)
(out_dir / "predictions").mkdir(parents=True, exist_ok=True)
(out_dir / "tokenizer").mkdir(parents=True, exist_ok=True)
(out_dir / "splits").mkdir(parents=True, exist_ok=True)

print(f"Root Directory      : {root}")
print(f>Data Directory       : {data_dir}")
print(f"Corpus Directory      : {corpus_dir}")
print(f"Classification CSV     : {cls_csv}")
print(f"Output Directory       : {out_dir}")
print(f"Checkpoints Directory  : {out_dir / 'checkpoints'}")

```

```

print(f"Generations Directory : {out_dir / 'generations'}")
print(f"Predictions Directory : {out_dir / 'predictions'}")
print(f"Tokenizer Directory    : {out_dir / 'tokenizer'}")
print(f"Splits Directory       : {out_dir / 'splits'}")

```

```

Root Directory      : f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3
Data Directory      : f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\Data
Corpus Directory    :
f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\Data\hindi_corpus\train
Classification CSV  : f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\Data\tex
t_classification_dataset\train.csv
Output Directory    : f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs
Checkpoints Directory :
f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs\checkpoints
Generations Directory :
f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs\generations
Predictions Directory :
f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs\predictions
Tokenizer Directory  :
f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs\tokenizer
Splits Directory     :
f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs\splits

```

0.5 PARAMS

```

[94]: VOCAB_SIZE = 5000
      BLOCK_SIZE = 192

      N_LAYER: int = 6
      N_HEAD: int = 6
      N_EMBED: int = 384
      DROPOUT: float = 0.1

      LM_BATCH_SIZE: int = 32
      LM_EPOCHS: int = 20
      LM_LEARNING_RATE: float = 3e-4
      LM_WARMUP_STEPS: int = 300
      LM_WEIGHT_DECAY: float = 0.1
      LM_LABEL_SMOOTHING: float = 0.1
      GRAD_CLIP: float = 1.0

      LM_MAX_TRAIN_STEPS_PER_EPOCH: int | None = 20000
      LM_MAX_VAL_STEPS_PER_EPOCH: int | None = 2000

      CLS_BATCH_SIZE: int = 32
      CLS_EPOCHS: int = 12
      CLS_LEARNING_RATE: float = 3e-5
      CLS_BACKBONE_LEARNING_RATE_MULTIPLIER: float = 0.2

```

```
CLS_WEIGHT_DECAY: float = 0.05
EARLY_STOPPING_PATIENCE: int = 3

VALIDATION_RATIO_LM: float = 0.1
VALIDATION_RATIO_CLS: float = 0.2
```

0.6 CORPUS PROCESSING

```
[95]: rows = []

for file in tqdm(sorted(corpus_dir.glob('*.txt')), desc="Reading Corpus "):
    try:
        txt = file.read_text(encoding='utf-8', errors='ignore').strip()
        if txt:
            rows.append({'filename': file.name.split('.')[0], 'text': txt})
    except Exception as e:
        continue

rows[:5]
```

Reading Corpus : 100%| | 44000/44000 [00:02<00:00, 15288.61it/s]

[illegible]


```
[97]: train_texts = [row['text'] for row in rows if row['filename'] in train_ids]
      val_texts = [row['text'] for row in rows if row['filename'] in val_ids]

[98]: (out_dir / 'splits' / 'lm_train_texts.txt').write_text('\n'.join(train_texts),
      ↪encoding='utf-8')
      (out_dir / 'splits' / 'lm_val_texts.txt').write_text('\n'.join(val_texts),
      ↪encoding='utf-8')

      print(f"Total Texts      : {len(rows)}")
      print(f"Training Texts   : {len(train_texts)}")
      print(f"Validation Texts  : {len(val_texts)}")
      print(f"Overlapping Texts : {len(train_ids.intersection(val_ids))}")
```

```
Total Texts      : 43331
Training Texts   : 38997
Validation Texts : 4334
Overlapping Texts : 0
```

0.7 SentencePiece TOKENIZER

```
[99]: # hindi words tokenization
      hindi_token_file = out_dir / 'tokenizer' / 'sp_train_text.txt'
      hindi_token_file.write_text('\n'.join(train_texts), encoding='utf-8')

      spm_prefix = out_dir / 'tokenizer' / 'hindi_spm_bpe'
      spm_model = spm_prefix.with_suffix('.model')

      if not spm_model.exists():
          spm.SentencePieceTrainer.train(
              input=str(hindi_token_file),
              model_prefix=str(spm_prefix),
              vocab_size=VOCAB_SIZE,
              model_type='bpe',
              character_coverage=0.9995,
              # max_sentence_length=10000,
              split_digits=False,
              unk_id=0,
              bos_id=1,
              eos_id=2,
              pad_id=3,
          )

      spm_processor = spm.SentencePieceProcessor(model_file=str(spm_model))

      print(f"Tokenizer Vocabulary Size      : {spm_processor.vocab_size()}")
      print(f"Tokenizer Pad ID                  : {spm_processor.pad_id()}")
      print(f"Tokenizer Unknown ID              : {spm_processor.unk_id()}")
      print(f"Tokenizer Beginning of Sentence ID : {spm_processor.bos_id()}")
```

```

print(f"Tokenizer End of Sentence ID      : {spm_processor.eos_id()}")
print(f"Tokenizer Sample Encoding        : {spm_processor.
    ↪ encode(train_texts[0][:100], out_type=int)[:20]}")
# print(f"Tokenizer Sample Encoding : {spm_processor.encode('
    ↪ out_type=str)}}")

```

```

Tokenizer Vocabulary Size      : 5000
Tokenizer Pad ID               : 3
Tokenizer Unknown ID          : 0
Tokenizer Beginning of Sentence ID : 1
Tokenizer End of Sentence ID    : 2
Tokenizer Sample Encoding      : [4203, 1499, 58, 2112, 1573, 3716, 4203,
11, 4875, 83, 58, 3887, 2569, 4203, 11, 4875]

```

0.8 DATA PROCESSING

```

[100]: class_df = pd.read_csv(cls_csv)
print(f"Columns : {list(class_df.columns)}, Row : {len(class_df)}")
class_df = class_df.dropna().copy()

if class_df['experience'].dtype == object:
    label_map = {"negative": 0, "neutral": 1, "positive": 2}
    class_df['label_id'] = class_df['experience'].astype(str).str.strip().str.
    ↪ lower().map(label_map)
else:
    class_df['label_id'] = class_df['experience'].astype(int)

class_df = class_df.dropna(subset=['label_id']).copy()
class_df['label_id'] = class_df['label_id'].astype(int)

class_train_df, class_val_df = train_test_split(class_df,
    ↪ test_size=VALIDATION_RATIO_CLS, random_state=SEED,
    ↪ stratify=class_df['label_id'])

print("Training set shape:", class_train_df.shape)
print("Validation set shape:", class_val_df.shape)
print("\nTraining set label counts:")
print(class_train_df['label_id'].value_counts().to_string())
print("\nValidation set label counts:")
print(class_val_df['label_id'].value_counts().to_string())

```

```

Columns : ['text', 'experience'], Row : 718
Training set shape: (574, 3)
Validation set shape: (144, 3)

```

```

Training set label counts:
label_id
2      218

```

```
0    192
1    164
```

Validation set label counts:

```
label_id
2     55
0     48
1     41
```

0.9 DATASET && DATALOADER PROCESSING

```
[101]: class LanguageModelingDataset(Dataset):
    def __init__(self, texts: List[str], block_size: int, eos_id: int):
        self.block_size = block_size
        token_lines = []

        for i in texts:
            token_lines.extend(spm_processor.encode(i, out_type=int))
            token_lines.append(eos_id)

        self.tokens = torch.tensor(token_lines, dtype=torch.long)

    def __len__(self):
        return max(0, len(self.tokens) - self.block_size - 1)

    def __getitem__(self, index):
        x = self.tokens[index : index + self.block_size]
        y = self.tokens[index + 1 : index + self.block_size + 1]
        return x, y

class TextClassificationDataset(Dataset):
    def __init__(self, frame: pd.DataFrame, text_col: str, max_len: int, pad_id:
        ↪ int):
        self.texts = frame[text_col].tolist()
        self.labels = frame['label_id'].tolist()
        self.max_len = max_len
        self.pad_id = pad_id

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, index):
        ids = spm_processor.encode(self.texts[index], out_type=int)[: self.
        ↪ max_len]
        if len(ids) < self.max_len:
            ids = ids + [self.pad_id] * (self.max_len - len(ids))
```

```

        return torch.tensor(ids, dtype=torch.long), torch.tensor(self.
        ↪labels[index], dtype=torch.long)

lm_train_dataset = LanguageModelingDataset(train_texts, block_size=BLOCK_SIZE,
        ↪eos_id=spm_processor.eos_id())
lm_val_dataset = LanguageModelingDataset(val_texts, block_size=BLOCK_SIZE,
        ↪eos_id=spm_processor.eos_id())

cls_train_dataset = TextClassificationDataset(class_train_df, text_col='text',
        ↪max_len=BLOCK_SIZE, pad_id=spm_processor.pad_id())
cls_val_dataset = TextClassificationDataset(class_val_df, text_col='text',
        ↪max_len=BLOCK_SIZE, pad_id=spm_processor.pad_id())

# Windows + notebooks + very large TensorDataset can crash with multiprocessing
        ↪workers.
if DEVICE.type == "cuda":
    LM_NUM_WORKERS = 0 if os.name == "nt" else min(4, os.cpu_count() or 1)
    CLS_NUM_WORKERS = 0 if os.name == "nt" else min(2, os.cpu_count() or 1)
else:
    LM_NUM_WORKERS = 0
    CLS_NUM_WORKERS = 0

PIN_MEMORY = DEVICE.type == "cuda"

lm_loader_kwargs = {
    "num_workers": LM_NUM_WORKERS,
    "pin_memory": PIN_MEMORY,
}
if LM_NUM_WORKERS > 0:
    lm_loader_kwargs.update({
        "persistent_workers": True,
        "prefetch_factor": 2,
    })

cls_loader_kwargs = {
    "num_workers": CLS_NUM_WORKERS,
    "pin_memory": PIN_MEMORY,
}
if CLS_NUM_WORKERS > 0:
    cls_loader_kwargs.update({
        "persistent_workers": True,
        "prefetch_factor": 2,
    })

```

```

lm_train_dataloader = DataLoader(lm_train_dataset, batch_size=LM_BATCH_SIZE,
    ↪shuffle=True, drop_last=True, **lm_loader_kwargs)
lm_val_dataloader = DataLoader(lm_val_dataset, batch_size=LM_BATCH_SIZE,
    ↪shuffle=False, drop_last=True, **lm_loader_kwargs)

cls_train_dataloader = DataLoader(cls_train_dataset, batch_size=CLS_BATCH_SIZE,
    ↪shuffle=True, **cls_loader_kwargs)
cls_val_dataloader = DataLoader(cls_val_dataset, batch_size=CLS_BATCH_SIZE,
    ↪shuffle=False, **cls_loader_kwargs)

print(f"Language Modeling\n- Training Batches    : {len(lm_train_dataloader)}\n-
    ↪Validation Batches : {len(lm_val_dataloader)}")
print(f"\nText Classification\n- Training Batches    :
    ↪{len(cls_train_dataloader)}\n- Validation Batches :
    ↪{len(cls_val_dataloader)}")
print(f"\nLoader setup\n- LM workers: {LM_NUM_WORKERS}\n- CLS workers:
    ↪{CLS_NUM_WORKERS}\n- pin_memory: {PIN_MEMORY}")

```

Language Modeling

```

- Training Batches    : 589908
- Validation Batches : 70460

```

Text Classification

```

- Training Batches    : 18
- Validation Batches : 5

```

Loader setup

```

- LM workers: 0
- CLS workers: 0
- pin_memory: True

```

0.10 MODEL

```

[102]: class MiniGPT(nn.Module):
    def __init__(self, vocab_size: int, block_size: int, n_layer: int, n_head:
    ↪int, n_embd: int, dropout: float):
        super().__init__()
        self.block_size = block_size
        self.token_embedding = nn.Embedding(vocab_size, n_embd)
        self.position_embedding = nn.Embedding(block_size, n_embd)
        self.dropout = nn.Dropout(dropout)

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=n_embd,
            nhead=n_head,
            dim_feedforward=4 * n_embd,
            dropout=dropout,

```

```

        activation="gelu",
        batch_first=True,
        norm_first=True,
    )

    self.transformer = nn.TransformerEncoder(
        encoder_layer,
        num_layers=n_layer,
        norm=nn.LayerNorm(n_embd),
        enable_nested_tensor=False,
    )

    def _causal_mask(self, T: int, device: torch.device, dtype: torch.dtype) -> torch.Tensor:
        mask = torch.full((T, T), float("-inf"), device=device, dtype=dtype)
        return torch.triu(mask, diagonal=1)

    def forward(self, x):
        _, T = x.shape
        if T > self.block_size:
            raise ValueError(f"Input sequence length {T} exceeds block size {self.block_size}")

        pos = torch.arange(0, T, device=x.device)
        h = self.token_embedding(x) + self.position_embedding(pos)[None, :, :]
        h = self.dropout(h)

        attention_mask = self._causal_mask(T, device=h.device, dtype=h.dtype)
        h = self.transformer(h, mask=attention_mask)
        return h

class GPTELanguageModel(nn.Module):
    def __init__(self, backbone: MiniGPT, vocab_size: int):
        super().__init__()
        self.backbone = backbone
        self.lm_head = nn.Linear(backbone.token_embedding.embedding_dim, vocab_size, bias=False)
        self.lm_head.weight = self.backbone.token_embedding.weight

    def forward(self, x, targets=None, label_smoothing=0.0):
        h = self.backbone(x)
        logits = self.lm_head(h)
        loss = None
        if targets is not None:
            B, T, V = logits.shape
            loss = nn.functional.cross_entropy(

```

```

        logits.reshape(B * T, V),
        targets.reshape(B * T),
        label_smoothing=label_smoothing,
    )
    return logits, loss

@torch.no_grad()
def generate(self, x, max_new_tokens=50, temperature=0.9, top_k=40):
    self.eval()

    for _ in range(max_new_tokens):
        x_cond = x[:, -self.backbone.block_size :]
        logits, _ = self.forward(x_cond)
        logits = logits[:, -1, :] / max(temperature, 1e-6)

        if top_k is not None and top_k > 0:
            k = min(top_k, logits.size(-1))
            values, _ = torch.topk(logits, k=k)
            logits[logits < values[:, [-1]]] = float("-inf")

        probs = torch.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1)
        x = torch.cat([x, next_token], dim=1)

    return x

class GPTTextClassifier(nn.Module):
    def __init__(self, backbone: MiniGPT, pad_id: int, num_classes: int = 3):
        super().__init__()
        self.backbone = backbone
        self.pad_id = pad_id
        self.dropout = nn.Dropout(0.4)
        self.fc = nn.Linear(backbone.token_embedding.embedding_dim, num_classes)

    def forward(self, x, targets=None):
        h = self.backbone(x)
        valid_mask = (x != self.pad_id).long()
        last_pos = (valid_mask.sum(dim=1) - 1).clamp(min=0)
        last_tok = h[torch.arange(h.size(0), device=h.device), last_pos]
        logits = self.fc(self.dropout(last_tok))
        loss = None

        if targets is not None:
            loss = nn.functional.cross_entropy(logits, targets)

    return logits, loss

```

```

[103]: backbone = MiniGPT(
    vocab_size=spm_processor.vocab_size(),
    block_size=BLOCK_SIZE,
    n_layer=N_LAYER,
    n_head=N_HEAD,
    n_embd=N_EMBED,
    dropout=DROPOUT,
).to(DEVICE)

lm_model = GPTLanguageModel(backbone, vocab_size=spm_processor.vocab_size()).
    ↪to(DEVICE)

lm_optimizer = GPTLanguageModel(backbone=backbone, vocab_size=spm_processor.
    ↪vocab_size()).to(DEVICE)

lm_optimizer = torch.optim.AdamW(
    lm_model.parameters(),
    lr=LM_LEARNING_RATE,
    betas=(0.9, 0.95),
    weight_decay=LM_WEIGHT_DECAY,
)

steps_per_epoch_raw = max(1, len(lm_train_dataloader))
steps_per_epoch = min(steps_per_epoch_raw, LM_MAX_TRAIN_STEPS_PER_EPOCH) if
    ↪LM_MAX_TRAIN_STEPS_PER_EPOCH is not None else steps_per_epoch_raw
totals_steps = LM_EPOCHS * steps_per_epoch

lr_scheduler = torch.optim.lr_scheduler.OneCycleLR(
    lm_optimizer,
    max_lr=LM_LEARNING_RATE,
    total_steps=totals_steps,
    pct_start=min(0.3, LM_WARMUP_STEPS / max(1, totals_steps)),
    anneal_strategy='cos',
)

scaler = torch.amp.GradScaler("cuda", enabled=USE_AMP)

print(f"LM scheduler steps/epoch: {steps_per_epoch} (raw:
    ↪{steps_per_epoch_raw})")
print(f"LM scheduler total steps: {totals_steps}")

```

```

LM scheduler steps/epoch: 20000 (raw: 589908)
LM scheduler total steps: 400000

```

0.11 LM MODEL SUMMARY

```
[104]: dummy_tokens = torch.zeros(2, 50, dtype=torch.long, device=DEVICE)

summary_1 = summary(
    backbone,
    input_data=dummy_tokens,
    dtypes=[torch.long],
    verbose=0,
    device=DEVICE,
)

summary_2 = summary(
    lm_model,
    input_data=dummy_tokens,
    dtypes=[torch.long],
    verbose=0,
    device=DEVICE,
)
```

```
[105]: print("\nBackbone Model Summary : ")
summary_1
```

Backbone Model Summary :

```
[105]: =====
=====
Layer (type:depth-idx)                Output Shape                Param #
=====
=====
MiniGPT                               [2, 50, 384]                --
  Embedding: 1-1                       [2, 50, 384]
1,920,000
  Embedding: 1-2                       [50, 384]                   73,728
  Dropout: 1-3                         [2, 50, 384]                --
  TransformerEncoder: 1-4              [2, 50, 384]                --
    ModuleList: 2-1                    --                           --
      TransformerEncoderLayer: 3-1      [2, 50, 384]
1,774,464
      TransformerEncoderLayer: 3-2      [2, 50, 384]
1,774,464
      TransformerEncoderLayer: 3-3      [2, 50, 384]
1,774,464
      TransformerEncoderLayer: 3-4      [2, 50, 384]
1,774,464
      TransformerEncoderLayer: 3-5      [2, 50, 384]
1,774,464
```

```

TransformerEncoderLayer: 3-6      [2, 50, 384]
1,774,464
LayerNorm: 2-2                    [2, 50, 384]      768
=====
=====
Total params: 12,641,280
Trainable params: 12,641,280
Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 21.73
=====
=====
Input size (MB): 0.00
Forward/backward pass size (MB): 13.67
Params size (MB): 36.37
Estimated Total Size (MB): 50.04
=====
=====

```

```
[106]: print("\n\nLM Model Summary : ")
summary_2
```

LM Model Summary :

```
[106]: =====
=====
Layer (type:depth-idx)      Output Shape
Param #
=====
=====
GPTLanguageModel           [2, 50, 5000]      --
  MiniGPT: 1-1              [2, 50, 384]       --
    Embedding: 2-1          [2, 50, 384]
1,920,000
    Embedding: 2-2          [50, 384]
73,728
    Dropout: 2-3            [2, 50, 384]       --
    TransformerEncoder: 2-4  [2, 50, 384]       --
      ModuleList: 3-1        --
10,646,784
        LayerNorm: 3-2       [2, 50, 384]       768
        Linear: 1-2          [2, 50, 5000]
1,920,000
=====
=====
Total params: 14,561,280
Trainable params: 14,561,280

```

Non-trainable params: 0
Total mult-adds (Units.MEGABYTES): 25.57

```
=====
=====
Input size (MB): 0.00
Forward/backward pass size (MB): 17.67
Params size (MB): 44.05
Estimated Total Size (MB): 61.72
=====
=====
```

```
[107]: os.makedirs("./outputs/Architecture", exist_ok=True)

dummy_tokens = torch.zeros(2, 50, dtype=torch.long, device=DEVICE)

model_graph = draw_graph(
    model=backbone,
    input_data=dummy_tokens,
    graph_name=backbone.__class__.__name__,
    expand_nested=True,
    save_graph=True,
    depth=10,
    directory="./outputs/Architecture"
)

gv.render(engine='dot', format='svg', filepath=f'./outputs/Architecture/
↳{backbone.__class__.__name__}.gv')
gv.render(engine='dot', format='pdf', filepath=f'./outputs/Architecture/
↳{backbone.__class__.__name__}.gv')
```

```
[107]: 'outputs\\Architecture\\MiniGPT.gv.pdf'
```

```
[108]: model_graph = draw_graph(
    model=lm_model,
    input_data=dummy_tokens,
    graph_name=lm_model.__class__.__name__,
    expand_nested=True,
    save_graph=True,
    depth=10,
    directory="./outputs/Architecture"
)

gv.render(engine='dot', format='svg', filepath=f'./outputs/Architecture/
↳{lm_model.__class__.__name__}.gv')
gv.render(engine='dot', format='pdf', filepath=f'./outputs/Architecture/
↳{lm_model.__class__.__name__}.gv')
```

```
[108]: 'outputs\\Architecture\\GPTLanguageModel.gv.pdf'
```

```
[20]: num_lm_epochs = LM_EPOCHS
lm_train_losses = np.zeros(num_lm_epochs)
lm_val_losses = np.zeros(num_lm_epochs)
lm_train_ppls = np.zeros(num_lm_epochs)
lm_val_ppls = np.zeros(num_lm_epochs)

# Reset scaler state on each notebook run to avoid stale optimizer-stage errors.
scaler = torch.amp.GradScaler("cuda", enabled=USE_AMP)
PIN_MEMORY = DEVICE.type == "cuda"

best_ppl = float("inf")
actual_lm_epochs = 0
lm_log = []
best_lm_path = out_dir / "checkpoints" / "lm_best.pt"
os.makedirs(best_lm_path.parent, exist_ok=True)

train_steps_limit = min(len(lm_train_dataloader), LM_MAX_TRAIN_STEPS_PER_EPOCH)
↳ if LM_MAX_TRAIN_STEPS_PER_EPOCH is not None else len(lm_train_dataloader)
val_steps_limit = min(len(lm_val_dataloader), LM_MAX_VAL_STEPS_PER_EPOCH) if
↳ LM_MAX_VAL_STEPS_PER_EPOCH is not None else len(lm_val_dataloader)
total_len = train_steps_limit + val_steps_limit

print(f"LM step limits -> train: {train_steps_limit}, val: {val_steps_limit}")

for epoch in range(num_lm_epochs):
    epoch_str = str(epoch + 1).rjust(len(str(num_lm_epochs)), " ")

    with tqdm(total=total_len, desc=f"LM Epoch [ {epoch_str}/{num_lm_epochs} ] :
↳ ") as pbar:
        lm_model.train()
        train_loss_list = []

        for step_idx, (x, y) in enumerate(lm_train_dataloader):
            if step_idx >= train_steps_limit:
                break

            x = x.to(DEVICE, non_blocking=PIN_MEMORY)
            y = y.to(DEVICE, non_blocking=PIN_MEMORY)
            lm_optimizer.zero_grad(set_to_none=True)

            if USE_AMP:
                with torch.amp.autocast("cuda", enabled=True):
                    _, loss = lm_model(x, y, label_smoothing=LM_LABEL_SMOOTHING)

            scaler.scale(loss).backward()
```

```

        scaler.unscale_(lm_optimizer)
        torch.nn.utils.clip_grad_norm_(lm_model.parameters(), GRAD_CLIP)
        scaler.step(lm_optimizer)
        scaler.update()
    else:
        _, loss = lm_model(x, y, label_smoothing=LM_LABEL_SMOOTHING)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(lm_model.parameters(), GRAD_CLIP)
        lm_optimizer.step()

    lr_scheduler.step()

    train_loss_list.append(loss.item())
    pbar.update(1)

    train_loss = float(np.mean(train_loss_list)) if train_loss_list else
↪float("inf")
    train_ppl = math.exp(train_loss) if train_loss < 20 else float("inf")

    lm_model.eval()
    val_loss_list = []
    with torch.no_grad():
        for step_idx, (x, y) in enumerate(lm_val_dataloader):
            if step_idx >= val_steps_limit:
                break

            x = x.to(DEVICE, non_blocking=PIN_MEMORY)
            y = y.to(DEVICE, non_blocking=PIN_MEMORY)

            with torch.amp.autocast("cuda", enabled=USE_AMP):
                _, vloss = lm_model(x, y, label_smoothing=0.0)

            val_loss_list.append(vloss.item())
            pbar.update(1)

    val_loss = float(np.mean(val_loss_list)) if val_loss_list else
↪float("inf")
    val_ppl = math.exp(val_loss) if val_loss < 20 else float("inf")

    lm_train_losses[epoch] = train_loss
    lm_val_losses[epoch] = val_loss
    lm_train_ppls[epoch] = train_ppl
    lm_val_ppls[epoch] = val_ppl
    actual_lm_epochs += 1

    pbar.set_description(f"LM Epoch [ {epoch_str}/{num_lm_epochs} ] ")
    pbar.set_postfix({

```

```

        "Train PPL": f"{train_ppl:.3f}",
        "Train loss": f"{train_loss:.3f}",
        "Val PPL": f"{val_ppl:.3f}",
        "Val loss": f"{val_loss:.3f}",
    })

    row = {
        "epoch": epoch + 1,
        "train_loss": train_loss,
        "train_ppl": train_ppl,
        "val_loss": val_loss,
        "val_ppl": val_ppl,
    }
    lm_log.append(row)

    # print(row)

    if val_ppl < best_ppl:
        best_ppl = val_ppl
        torch.save(lm_model.state_dict(), best_lm_path)

print(f"\nLM training complete. Best Val PPL = {best_ppl:.4f}")

```

LM step limits -> train: 20000, val: 2000

```

LM Epoch [ 1/20 ]: 100%|          | 22000/22000 [13:29<00:00, 27.17it/s, Train
PPL=592.683, Train loss=6.385, Val PPL=58.921, Val loss=4.076]
LM Epoch [ 2/20 ]: 100%|          | 22000/22000 [13:34<00:00, 27.02it/s, Train
PPL=100.625, Train loss=4.611, Val PPL=40.819, Val loss=3.709]
LM Epoch [ 3/20 ]: 100%|          | 22000/22000 [13:34<00:00, 27.01it/s, Train
PPL=79.375, Train loss=4.374, Val PPL=36.086, Val loss=3.586]
LM Epoch [ 4/20 ]: 100%|          | 22000/22000 [13:35<00:00, 26.96it/s, Train
PPL=71.384, Train loss=4.268, Val PPL=33.574, Val loss=3.514]
LM Epoch [ 5/20 ]: 100%|          | 22000/22000 [13:35<00:00, 26.98it/s, Train
PPL=67.536, Train loss=4.213, Val PPL=32.450, Val loss=3.480]
LM Epoch [ 6/20 ]: 100%|          | 22000/22000 [13:39<00:00, 26.85it/s, Train
PPL=65.183, Train loss=4.177, Val PPL=31.284, Val loss=3.443]
LM Epoch [ 7/20 ]: 100%|          | 22000/22000 [13:21<00:00, 27.47it/s, Train
PPL=63.454, Train loss=4.150, Val PPL=30.956, Val loss=3.433]
LM Epoch [ 8/20 ]: 100%|          | 22000/22000 [13:31<00:00, 27.12it/s, Train
PPL=61.817, Train loss=4.124, Val PPL=30.173, Val loss=3.407]
LM Epoch [ 9/20 ]: 100%|          | 22000/22000 [13:39<00:00, 26.86it/s, Train
PPL=60.441, Train loss=4.102, Val PPL=29.950, Val loss=3.400]
LM Epoch [ 10/20 ]: 100%|         | 22000/22000 [13:18<00:00, 27.54it/s, Train
PPL=59.099, Train loss=4.079, Val PPL=28.833, Val loss=3.362]
LM Epoch [ 11/20 ]: 100%|         | 22000/22000 [13:17<00:00, 27.58it/s, Train
PPL=57.855, Train loss=4.058, Val PPL=28.548, Val loss=3.352]
LM Epoch [ 12/20 ]: 100%|         | 22000/22000 [13:19<00:00, 27.52it/s, Train

```

```
PPL=56.736, Train loss=4.038, Val PPL=27.904, Val loss=3.329]
LM Epoch [ 13/20 ]: 100%|      | 22000/22000 [13:41<00:00, 26.77it/s, Train
PPL=55.489, Train loss=4.016, Val PPL=27.361, Val loss=3.309]
LM Epoch [ 14/20 ]: 100%|      | 22000/22000 [13:41<00:00, 26.79it/s, Train
PPL=54.412, Train loss=3.997, Val PPL=26.951, Val loss=3.294]
LM Epoch [ 15/20 ]: 100%|      | 22000/22000 [13:47<00:00, 26.59it/s, Train
PPL=53.350, Train loss=3.977, Val PPL=26.604, Val loss=3.281]
LM Epoch [ 16/20 ]: 100%|      | 22000/22000 [14:07<00:00, 25.96it/s, Train
PPL=52.487, Train loss=3.961, Val PPL=26.235, Val loss=3.267]
LM Epoch [ 17/20 ]: 100%|      | 22000/22000 [13:38<00:00, 26.88it/s, Train
PPL=51.684, Train loss=3.945, Val PPL=26.032, Val loss=3.259]
LM Epoch [ 18/20 ]: 100%|      | 22000/22000 [13:38<00:00, 26.89it/s, Train
PPL=51.222, Train loss=3.936, Val PPL=25.763, Val loss=3.249]
LM Epoch [ 19/20 ]: 100%|      | 22000/22000 [13:37<00:00, 26.92it/s, Train
PPL=50.782, Train loss=3.928, Val PPL=25.660, Val loss=3.245]
LM Epoch [ 20/20 ]: 100%|      | 22000/22000 [13:38<00:00, 26.88it/s, Train
PPL=50.678, Train loss=3.925, Val PPL=25.646, Val loss=3.244]
```

LM training complete. Best Val PPL = 25.6464

```
[ ]: print("Best val perplexity      : ", best_ppl)
      print("Saved best LM checkpoint : ", best_lm_path)
      pd.DataFrame(lm_log)
```

Best val perplexity : 25.64636466148991

Saved best LM checkpoint :

f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs\checkpoints\lm_best.pt

```
[ ]:      epoch  train_loss  train_ppl  val_loss  val_ppl
0         1    6.384661  592.683494  4.076190  58.920580
1         2    4.611404  100.625353  3.709145  40.818902
2         3    4.374181   79.374840  3.585917  36.086424
3         4    4.268074   71.384018  3.513753  33.574036
4         5    4.212661   67.536035  3.479713  32.450393
5         6    4.177202   65.183198  3.443097  31.283706
6         7    4.150316   63.454017  3.432580  30.956413
7         8    4.124179   61.817062  3.406956  30.173251
8         9    4.101675   60.441441  3.399538  29.950262
9        10    4.079214   59.099003  3.361505  28.832555
10       11    4.057933   57.854614  3.351576  28.547684
11       12    4.038413   56.736215  3.328777  27.904184
12       13    4.016184   55.488968  3.309114  27.360864
13       14    3.996589   54.412232  3.294008  26.950657
14       15    3.976871   53.349831  3.281046  26.603597
15       16    3.960562   52.486838  3.267109  26.235388
16       17    3.945147   51.683944  3.259323  26.031903
17       18    3.936164   51.221735  3.248954  25.763385
```

18	19	3.927535	50.781636	3.244919	25.659631
19	20	3.925499	50.678383	3.244402	25.646365

```
[109]: lm_model.load_state_dict(torch.load(best_lm_path, map_location=DEVICE))

prompt = val_texts[1][:200] if len(val_texts) > 0 else " "
prompt_ids = torch.tensor([spm_processor.encode(prompt, out_type=int)],
    dtype=torch.long, device=DEVICE)

with torch.no_grad():
    out_ids = lm_model.generate(prompt_ids, max_new_tokens=50, temperature=0.
    85, top_k=50).squeeze(0).tolist()

generated_text = spm_processor.decode(out_ids)
gen_file = out_dir / "generations" / "generated_hindi.txt"
gen_file.write_text(generated_text, encoding="utf-8")

print("Prompt          : ", prompt)
print("Generated        : ", generated_text[:700])
print("\nSaved generation : ", gen_file)
```

Prompt : 1867 .

Generated : 1867 .

-

,

Saved generation : f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs\generations\generated_hindi.txt

```
[110]: lm_model.load_state_dict(torch.load(best_lm_path, map_location=DEVICE))

classifier = GPTTextClassifier(
    backbone=lm_model.backbone,
    pad_id=spm_processor.pad_id(),
    num_classes=3,
).to(DEVICE)

cls_counts = class_train_df["label_id"].value_counts().sort_index()
weights = torch.tensor([1.0 / max(1, cls_counts.get(i, 1)) for i in range(3)],
    dtype=torch.float32, device=DEVICE)
weights = weights / weights.sum() * 3.0

backbone_params = list(classifier.backbone.parameters())
```

```

head_params = list(classifier.fc.parameters())
cls_optimizer = torch.optim.AdamW(
    [
        {"params": backbone_params, "lr": CLS_LEARNING_RATE *
↪CLS_BACKBONE_LEARNING_RATE_MULTIPLIER},
        {"params": head_params, "lr": CLS_LEARNING_RATE},
    ],
    weight_decay=CLS_WEIGHT_DECAY,
)
cls_scaler = torch.amp.GradScaler("cuda", enabled=USE_AMP)

for p in classifier.backbone.parameters():
    p.requires_grad = True
freeze_blocks = max(1, N_LAYER // 2)
for bi, block in enumerate(classifier.backbone.transformer.layers):
    if bi < freeze_blocks:
        for p in block.parameters():
            p.requires_grad = False

```

```

[111]: from sklearn.metrics import f1_score

num_cls_epochs = CLS_EPOCHS
cls_train_losses = np.zeros(num_cls_epochs)
cls_val_losses = np.zeros(num_cls_epochs)
cls_train_f1s = np.zeros(num_cls_epochs)
cls_val_f1s = np.zeros(num_cls_epochs)
cls_train_accs = np.zeros(num_cls_epochs)
cls_val_accs = np.zeros(num_cls_epochs)

best_acc = -1.0
best_f1 = -1.0
patience = 0
actual_cls_epochs = 0
cls_log = []
best_y_true, best_y_pred = [], []

best_cls_path = out_dir / "checkpoints" / "classifier_best.pt"
cls_scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    cls_optimizer,
    mode="max",
    factor=0.5,
    patience=1,
    min_lr=1e-6,
)

total_len = len(cls_train_dataloader) + len(cls_val_dataloader)

```

```

for epoch in range(num_cls_epochs):
    if epoch == 2:
        for p in classifier.backbone.parameters():
            p.requires_grad = True

    epoch_str = str(epoch + 1).rjust(len(str(num_cls_epochs)), " ")
    with tqdm(total=total_len, desc=f"CLS Epoch [ {epoch_str}/{num_cls_epochs}"]
↪ : ") as pbar:
        classifier.train()
        train_loss_list = []
        train_true, train_pred = [], []

        for x, y in cls_train_dataloader:
            x = x.to(DEVICE, non_blocking=PIN_MEMORY)
            y = y.to(DEVICE, non_blocking=PIN_MEMORY)

            with torch.amp.autocast("cuda", enabled=USE_AMP):
                logits, _ = classifier(x, None)
                loss = nn.functional.cross_entropy(logits, y, weight=weights)

            cls_optimizer.zero_grad(set_to_none=True)
            cls_scaler.scale(loss).backward()
            cls_scaler.unscale_(cls_optimizer)
            torch.nn.utils.clip_grad_norm_(classifier.parameters(), GRAD_CLIP)
            cls_scaler.step(cls_optimizer)
            cls_scaler.update()

            train_loss_list.append(loss.item())
            train_pred.extend(logits.argmax(dim=-1).detach().cpu().tolist())
            train_true.extend(y.detach().cpu().tolist())
            pbar.update(1)

        train_loss = float(np.mean(train_loss_list)) if train_loss_list else
↪ float("inf")
        train_acc = accuracy_score(train_true, train_pred) if train_true else 0.
↪ 0
        train_f1 = f1_score(train_true, train_pred, average="macro",
↪ zero_division=0) if train_true else 0.0

        classifier.eval()
        val_loss_list = []
        y_true, y_pred = [], []

        with torch.no_grad():
            for x, y in cls_val_dataloader:
                x = x.to(DEVICE, non_blocking=PIN_MEMORY)
                y = y.to(DEVICE, non_blocking=PIN_MEMORY)

```

```

        with torch.amp.autocast("cuda", enabled=USE_AMP):
            logits, _ = classifier(x, None)
            vloss = nn.functional.cross_entropy(logits, y,
↪weight=weights)

            val_loss_list.append(vloss.item())
            y_pred.extend(logits.argmax(dim=-1).cpu().tolist())
            y_true.extend(y.cpu().tolist())
            pbar.update(1)

    val_loss = float(np.mean(val_loss_list)) if val_loss_list else
↪float("inf")
    val_acc = accuracy_score(y_true, y_pred) if y_true else 0.0
    val_f1 = f1_score(y_true, y_pred, average="macro", zero_division=0) if
↪y_true else 0.0

    cls_train_losses[epoch] = train_loss
    cls_val_losses[epoch] = val_loss
    cls_train_f1s[epoch] = train_f1
    cls_val_f1s[epoch] = val_f1
    cls_train_accs[epoch] = train_acc
    cls_val_accs[epoch] = val_acc
    actual_cls_epochs += 1

    pbar.set_description(f"CLS Epoch [ {epoch_str}/{num_cls_epochs} ] ")
    pbar.set_postfix({
        "Train F1": f"{train_f1:.3f}",
        "Train loss": f"{train_loss:.3f}",
        "Train Acc": f"{train_acc:.3f}",
        "Val F1": f"{val_f1:.3f}",
        "Val loss": f"{val_loss:.3f}",
        "Val Acc": f"{val_acc:.3f}",
    })

    cls_scheduler.step(val_f1)

    row = {
        "epoch": epoch + 1,
        "train_loss": train_loss,
        "train_acc": train_acc,
        "train_f1": train_f1,
        "val_loss": val_loss,
        "val_acc": val_acc,
        "val_f1": val_f1,
    }
    cls_log.append(row)

```

```

# print(row)

if val_f1 > best_f1:
    best_f1 = val_f1
    best_acc = max(best_acc, val_acc)
    patience = 0
    best_y_true, best_y_pred = list(y_true), list(y_pred)
    torch.save(classifier.state_dict(), best_cls_path)
else:
    best_acc = max(best_acc, val_acc)
    patience += 1
    if patience >= EARLY_STOPPING_PATIENCE:
        print(f"Early stopping triggered at epoch {epoch + 1}")
        break

if not best_y_true:
    best_y_true, best_y_pred = list(y_true), list(y_pred)
y_true, y_pred = best_y_true, best_y_pred

best_epoch_idx = int(np.argmax(cls_val_f1s[:actual_cls_epochs])) if
↳actual_cls_epochs > 0 else 0
best_train_acc = float(cls_train_accs[best_epoch_idx]) if actual_cls_epochs > 0
↳else 0.0
best_val_acc = float(cls_val_accs[best_epoch_idx]) if actual_cls_epochs > 0
↳else 0.0

print(f"\nClassifier training complete. Best Val F1 = {best_f1:.4f}")
print(f"Train Accuracy (best epoch) : {best_train_acc:.4f}")
print(f"Val Accuracy (best epoch) : {best_val_acc:.4f}")

```

```

CLS Epoch [ 1/12 ] : 100%|      | 23/23 [00:01<00:00, 19.29it/s, Train
F1=0.381, Train loss=2.121, Train Acc=0.383, Val F1=0.367, Val loss=1.739, Val
Acc=0.368]
CLS Epoch [ 2/12 ] : 100%|      | 23/23 [00:01<00:00, 19.58it/s, Train
F1=0.342, Train loss=2.051, Train Acc=0.343, Val F1=0.358, Val loss=1.704, Val
Acc=0.361]
CLS Epoch [ 3/12 ] : 100%|      | 23/23 [00:01<00:00, 18.30it/s, Train
F1=0.384, Train loss=2.006, Train Acc=0.385, Val F1=0.381, Val loss=1.664, Val
Acc=0.382]
CLS Epoch [ 4/12 ] : 100%|      | 23/23 [00:01<00:00, 18.30it/s, Train
F1=0.368, Train loss=1.775, Train Acc=0.369, Val F1=0.344, Val loss=1.617, Val
Acc=0.347]
CLS Epoch [ 5/12 ] : 100%|      | 23/23 [00:01<00:00, 18.20it/s, Train
F1=0.366, Train loss=1.762, Train Acc=0.366, Val F1=0.389, Val loss=1.593, Val
Acc=0.389]
CLS Epoch [ 6/12 ] : 100%|      | 23/23 [00:01<00:00, 18.43it/s, Train
F1=0.395, Train loss=1.579, Train Acc=0.395, Val F1=0.388, Val loss=1.558, Val

```

```

Acc=0.389]
CLS Epoch [ 7/12 ] : 100%|      | 23/23 [00:01<00:00, 18.24it/s, Train
F1=0.431, Train loss=1.445, Train Acc=0.432, Val F1=0.368, Val loss=1.527, Val
Acc=0.368]
CLS Epoch [ 8/12 ] : 100%|      | 23/23 [00:01<00:00, 18.00it/s, Train
F1=0.428, Train loss=1.456, Train Acc=0.429, Val F1=0.375, Val loss=1.510, Val
Acc=0.375]

```

Early stopping triggered at epoch 8

Classifier training complete. Best Val F1 = 0.3889

Train Accuracy (best epoch) : 0.3659

Val Accuracy (best epoch) : 0.3889

```

[112]: print("Best validation F1          : ", best_f1)
       print("Best validation accuracy      : ", best_acc)
       print("Saved best classifier checkpoint : ", best_cls_path)
       print("\nValidation Classification Report (best epoch) : ")
       print(classification_report(y_true, y_pred, digits=4))
       print("Confusion Matrix : \n", confusion_matrix(y_true, y_pred))

```

Best validation F1 : 0.38892283045508846

Best validation accuracy : 0.3888888888888889

Saved best classifier checkpoint : f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs\checkpoints\classifier_best.pt

Validation Classification Report (best epoch) :

	precision	recall	f1-score	support
0	0.4000	0.3750	0.3871	48
1	0.3276	0.4634	0.3838	41
2	0.4634	0.3455	0.3958	55
accuracy			0.3889	144
macro avg	0.3970	0.3946	0.3889	144
weighted avg	0.4036	0.3889	0.3895	144

Confusion Matrix :

```
[[18 21  9]
```

```
[ 9 19 13]
```

```
[18 18 19]]
```

```
[113]: pd.DataFrame(cls_log)
```

```

[113]:   epoch  train_loss  train_acc  train_f1  val_loss  val_acc  val_f1
0      1    2.120932    0.383275  0.380846  1.739382  0.368056  0.367178
1      2    2.050513    0.343206  0.342103  1.703725  0.361111  0.358218

```

2	3	2.006447	0.385017	0.384321	1.663564	0.381944	0.380587
3	4	1.775146	0.369338	0.367966	1.616755	0.347222	0.344187
4	5	1.761770	0.365854	0.365620	1.592973	0.388889	0.388923
5	6	1.579164	0.395470	0.394719	1.558197	0.388889	0.387514
6	7	1.444500	0.432056	0.431384	1.526779	0.368056	0.368473
7	8	1.455663	0.428571	0.427961	1.510417	0.375000	0.374956

```
[ ]: classifier.load_state_dict(torch.load(best_cls_path, map_location=DEVICE))
classifier.eval()

cls_val_ds_ref = cls_val_dataset if "cls_val_dataset" in globals() else
↳cls_val_ds
cls_val_df_ref = class_val_df if "class_val_df" in globals() else cls_val_df
text_col_ref = "text" if "text" in cls_val_df_ref.columns else (TEXT_COL if
↳"TEXT_COL" in globals() else cls_val_df_ref.columns[0])
out_root = out_dir if "out_dir" in globals() else OUT

correct_rows = []
with torch.no_grad():
    for i in range(len(cls_val_ds_ref)):
        x, y = cls_val_ds_ref[i]
        logits, _ = classifier(x.unsqueeze(0).to(DEVICE), None)
        pred = int(logits.argmax(dim=-1).item())
        gold = int(y.item())
        if pred == gold:
            text = str(cls_val_df_ref.iloc[i][text_col_ref])
            correct_rows.append((text, pred, gold))
        if len(correct_rows) >= 5:
            break

id2label = {0: "Negative", 1: "Neutral", 2: "Positive"}
correct_file = out_root / "predictions" / "5_correct_samples.txt"
with correct_file.open("w", encoding="utf-8") as f:
    for j, (text, pred, gold) in enumerate(correct_rows, 1):
        f.write(f"Sample {j}\n")
        f.write(f"Predicted: {id2label[pred]} ({pred})\n")
        f.write(f"Gold: {id2label[gold]} ({gold})\n")
        f.write("Text:\n")
        f.write(text.strip() + "\n")
        f.write("-" * 80 + "\n")

print("Saved correct-sample file : ", correct_file)
print("Count saved : ", len(correct_rows))
```

Saved correct-sample file: f:\Sayan\Study\Prog\Prog\ML_AI\NLP_assignment_3\outputs\predictions\5_correct_samples.txt
Count saved: 5

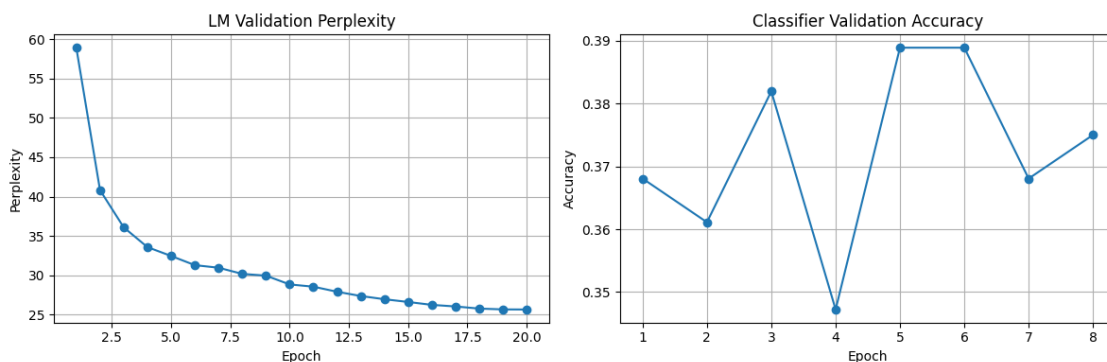
```
[115]: lm_df = pd.DataFrame(lm_log)
cls_df_log = pd.DataFrame(cls_log)

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
if len(lm_df) > 0:
    ax[0].plot(lm_df["epoch"], lm_df["val_ppl"], marker="o")
    ax[0].set_title("LM Validation Perplexity")
    ax[0].set_xlabel("Epoch")
    ax[0].set_ylabel("Perplexity")
    ax[0].grid(True)

if len(cls_df_log) > 0:
    ax[1].plot(cls_df_log["epoch"], cls_df_log["val_acc"], marker="o")
    ax[1].set_title("Classifier Validation Accuracy")
    ax[1].set_xlabel("Epoch")
    ax[1].set_ylabel("Accuracy")
    ax[1].grid(True)

plt.tight_layout()
plt.show()

summary = {
    "best_val_ppl": float(min([r["val_ppl"] for r in lm_log])) if lm_log else
    ↪None,
    "best_val_acc": float(max([r["val_acc"] for r in cls_log])) if cls_log else
    ↪None,
    "lm_ckpt": str(best_lm_path),
    "cls_ckpt": str(best_cls_path),
    "generation_file": str(gen_file),
    "correct_samples_file": str(correct_file),
}
summary
```



```
[115]: {'best_val_ppl': 25.64636466148991,  
        'best_val_acc': 0.3888888888888889,  
        'lm_ckpt': 'f:\\Sayan\\Study\\Prog\\Prog\\ML_AI\\NLP_assignment_3\\outputs\\che  
ckpoints\\lm_best.pt',  
        'cls_ckpt': 'f:\\Sayan\\Study\\Prog\\Prog\\ML_AI\\NLP_assignment_3\\outputs\\ch  
eckpoints\\classifier_best.pt',  
        'generation_file': 'f:\\Sayan\\Study\\Prog\\Prog\\ML_AI\\NLP_assignment_3\\outp  
uts\\generations\\generated_hindi.txt',  
        'correct_samples_file': 'f:\\Sayan\\Study\\Prog\\Prog\\ML_AI\\NLP_assignment_3\\  
outputs\\predictions\\5_correct_samples.txt'}
```

```
[ ]:
```